HP601A—Examinationsuppgift

Calle Ketola

16 mars 2026

Innehåll

1 Inledning **1**

2 Det valda kursmomentet **1**
2.1 Konstruktiv länkning 2
2.2 Likabehandling 4
2.3 Policy och styrdokument 5

3 Det nya kursmomentet **6**
3.1 Förändringar 7
3.2 Konstruktiv länkning 8
3.3 Likabehandling 8
3.4 Policy och styrdokument 9

4 Hur AI har använts i den här texten **10**

Referenser **11**

A Antagna och sökande hösten 2025 **13**

B Genomströmning på program med inriktning Elektronik, datateknik och automation **13**

C Tidigare version av uppgiften **14**

D Uppdaterad version av uppgiften **21**

1 Inledning

I detta paper diskuteras ett kursmoment i en kurs som nyligen getts vid Malmö universitet. Momenten analyseras utifrån hur det relaterar till tre punkter, konstruktiv länkning, likabehandling samt styrdokument samt tre perspektiv, student-, lärar- och samhällsperspektiv. Det ges även ett förslag på hur detta moment kan designas om för att bättre nå lärandemålen för kursen och utbildningen som helhet.

Det kursmoment som jag har valt att analysera är den första workshopen i kursen *CD102A Objektorienterad programmering* som läses i läsperiod två det första läsåret utav programmet *Civilingenjör i Datateknik* vid Malmö universitet. Anledningen till att jag valt detta moment är för att när vi genomförde momentet hösten 2025 blev studenterna stressade och workshopen drog över tiden med ungefär 45 minuter. En anledning till att workshopen drog över tiden var för att bedömningarna tog längre tid än planerat.

En anledning till att jag velat förändra uppgiften är att om man följer Feisel-Schmittz taxonomi så ligger ”att bara lösa uppgiften” lågt på skalan av enkelhet och komplexitet (Elmgren & Olding, 2025, sida 159). Målet i kursen, och utbildningen, är att studenterna ska kunna analysera och utvärdera olika lösningar (Malmö universitet, 2025a; Malmö universitet, 2025b) vilket ligger överst i Feisel-Schmittz taxonomi vilket är vad jag vill jobba mot i detta moment.

2 Det valda kursmomentet

I det valda momentet fick studenterna i uppgift att två-och-två skapa ett program utifrån en uppgiftsbeskrivning, samt skapa ett klassdiagram som modellerade deras program. I programmet skulle de skapa tre klasser, klassen `Galaxy`, klassen `Star`, samt klassen `Planet`. Till sin hjälp att göra detta fick studenterna beskrivningar av varje klass, vilka attribut de skulle innehålla, samt korta beskrivningar över vilka operationer klasserna skulle ha.

För att kontrollera att de var färdiga hade studenterna fått en *driver*-kod som testade funktionaliteten i deras lösningar. När den skrev ut ”*Present your solution*” så fick de lov att redovisa sin lösning för en lärare.

Under workshopen blev ca 90 % av de deltagande studenterna godkända. Då bedömningarna drog över tiden, är det oklart hur många som faktiskt var färdiga när passets egentliga stopptid hade nåtts. För att bli godkända skulle studenterna kunna förklara sina lösningar och visa upp ett korrekt klassdiagram som visade hur klasserna relaterade till varandra.

Under ett uppföljande moment veckan efter fick studenterna möjlighet att bygga vidare på sina lösningar från detta moment. Då var det flera som hade problem att göra ändringar i sin kod vilket tyder på att det studenterna hade gjort under det tidigare momentet inte hade befästs tillräckligt för att jobba vidare med (Elmgren & Olding, 2025, sida 22). Detta ser jag som ett bevis för att studenternas kunskap här ligger lågt i Feisel-Schmittz taxonomi (Elmgren & Olding, 2025, sida 159).

2.1 Konstruktiv länkning

Detta är kursens första bedömda moment, och sätter således tonen för kommande moment. Inför detta moment har studenterna haft fem föreläsningar och fyra övningstillfällen. De första två övningstillfällena har behandlat skillnader mellan programspråken Java, som de ska använda i den här kursen, och Python, som de har använt i den föregående kursen *CD100A Imperativ programmering*. Under tredje tillfället fick studenterna en uppgift där klassbeskrivningarna har getts på ett liknande sätt som i momentet vi ska undersöka, och under det fjärde övningstillfället har de fått använda ett verktyg för att skapa klassdiagram.

Den här workshopen bygger således vidare på övningstillfälle tre och fyra. Skillnaderna är dels att de nu ska göra en kombination av de båda tidigare övningarna samt att programmet testar sig själv. För att kontrollera att programmet var redo att presenteras skulle studenterna köra den *driver*-kod som de fick till uppgiften. Denna kod utförde tester på studenternas kod och skrev ut vad det försökte göra, det förväntade svaret, samt det erhållna svaret från studentens kod. Detta lät studenterna snabbt se om något inte var rätt, på ett ungefär var det var fel, och ungefär vad som var fel. Detta möjliggjorde och förenklade till viss del en självbedömning hos studenterna (Elmgren & Olding, 2025, sida 278). Det underlättade även för lärarna då studenterna i samma utsträckning inte behövde be läraren kontrollera om svaret var korrekt. Rättningen i slutet av passet underlättades också eftersom programmet berättade att det fungerade som det skulle.

I de två kommande bedömda workshoparna är den tematiska inramningen den samma, med stjärnor och planeter, men uppgifterna ändrar karaktär. De andra workshoparna saknade självrättande delar och mängden kod som behövde skrivas var mindre. Att mängden kod som behövde skrivas minskades berodde framförallt på att den första workshopen drog över på tiden. Således var den konstruktiva länkningen inte särdeles stark till dessa moment.

Kopplingen var desto starkare till de två bedömda inlämningsuppgifterna som följde. Framförallt eftersom de tekniker de uppmuntrades att använda i det första momentet kunde användas för att lösa problem som uppstod i de båda inlämningsuppgifterna. Det var särskilt tekniker för att hämta information ur objekt som låg i andra objekt, samt för att hantera arrayer med objekt som de hade användning av.

När tentamen kom så återanvände vi delar av modellen för stjärnor och planeter som vi använt i workshopen. Detta ledde till att flera studenter gav mer utförliga svar än vad som krävdes för godkända poäng.

Ett problem som kan ha uppstått är att eftersom det här momentet blev så stressigt är att studenterna leddes in i en mentalitet där själva skrivandet av kod är det viktigaste och att kvantitet — som i att skriva kod snabbt — är viktigare än kvalitet — som i att skriva rätt kod och utvärdera valet av lösning.

2.1.1 Studentperspektiv

För studenter som har som mål att bli civilingenjörer och få ett ”bra” jobb, de som av John Biggs symboliseras av Robert (Biggs m. fl., 2022, sida 16), har uppgiften uppskattats. Detta vet vi efter samtal med studenter efter kursens slut. För studenterna som är mer som Susan, och mer är ute efter kunskapen för kunskapens skull, har uppgiften varit stressande på grund av att de inte har

hunnit förstå hur deras lösningar fungerar. Sen har vi en tredje kategori studenter som består av dem 10 % som inte blev godkända. För dessa studenterna var uppgiften för stor och för svår. Det är oklart om de hade klarat uppgiften om de fått ännu mer tid än de extra 45 minuterna som det blev. Att några studenter inte klarar uppgiften är inte fel i sig, särskilt som inte alla i gruppen var godkända på alla moment från föregående kurser.

2.1.2 Lärarperspektiv

När uppgiften skapades var tanken att den skulle:

1. knyta samman det som gjorts i kursen hittills,
2. knyta an till den senaste föreläsningen,
3. vara lätt att bedöma.

Uppgiften var således inte nämnvärt framåtriktad, istället var fokus på vad som hade gjorts tidigare. Detta kan ge en konstruktiv länkning till tidigare material, men inte nödvändigtvis till lärandemålen. Dock knyter uppgiften an till lärande mål sju, samt till viss del till lärandemål fyra och åtta (Malmö universitet, 2025a). På så vis kan man se en konstruktiv länkning mellan uppgiften och lärandemålen.

Uppgiften blev inte svår att bedöma, men den tog tid att bedöma. Att avgöra om en student förstår sin lösning är en tidskrävande process. Samtidigt kan studentens förståelse fördjupas när hen får sätta ord på vad hen har gjort (Elmgren & Olding, 2025, sida 175).

2.1.3 Samhällsperspektiv

Under den här kursens gång har en kritik mot konstruktiv länkning som lyfts varit att om innehållet i föreläsningar och övningar för starkt kopplas till examinerande moment, alltså om man undervisar det som ska examineras och examinerar det så som det har undervisats, så finns det en risk att studenterna får en smal kunskap och en falsk tilltro till instruktioner.

En mem inom lärarkåren är studenter och elever som ställer frågan ”kommer det här på provet/tentan”. Skulle läraren svara nej så tappar studenterna intresset för det som diskuteras. I stunden kan det för studenten vara ett rationellt beslut att inte ta emot den nya informationen för att minska mängden information som behöver bearbetas. Men i längden går studenten miste om information som hade kunnat vara viktig utanför examinationens ramar. Detta är ett tecken på att studenterna ser undervisningen som ett verktyg för att få betyg, betyget är sen ett verktyg för att få examen, och examen ett verktyg för att få ett jobb. Detta indikerar att studenterna ser högskolan som en *kunskapsfabrik* där skolan ska producera så många jobbredo individer som möjligt (Brandén, 2022, sida 274). En möjlig risk med detta är att högskolan producerar individer som kan lösa problem som är snarlika det dem har stött på under sin utbildning, men inte problem som är ”nya”.

Att lägga ifrån sig pennan när studenten får höra att det inte kommer på tentan är också ett tecken på att studenten ägnar sig åt ytinriktat lärande (Elmgren & Olding, 2025, sida 22). Detta är inte målet med utbildningen, istället

vill vi har studenter som istället för att ha memorerat fakta och lösningsmetoder har förstått hur saker hänger ihop och hur det fungerar, så kallat djupinlärning (Elmgren & Olding, 2025, sida 22).

2.2 Likabehandling

Som civilingenjörsstudenter har studenterna med sig kurserna Fysik 1 och Fysik 2 från gymnasiet, eller Komvux. I dessa kurser får de bland annat lära sig om universum och astronomi (Ämnesplan Fysik, 2010). Det borde därför inte vara något i bakgrunden till uppgiften som inte alla redan har en tidigare förståelse för. Studenterna har också läst en introducerande kurs i programmering som de förväntas ha goda kunskaper från, även om de ännu inte är godkända.

Det är fler kvinnor än män som söker till högskoleutbildningar (Universitets- och högskolerådet, 2025), men utbildningar med inriktning på ”data” har färre kvinnliga sökande än män (se bilaga A). Det är ungefär lika stor andel av de antagna männen som kvinnorna som klarar en utbildning med inritning på elektronik, datateknik, eller automation (se bilaga B). Jag ser inte heller något i hur uppgiften läggs upp som skulle gynna eller missgynna antingen män eller kvinnor. Uppgiften är också frånkopplad från frågor rörande människor i allmänhet. Den mänskliga kopplingen i uppgiften är att studenterna förväntades att jobba i par, där de själva fick välja vem de skulle jobba med. Skulle några studenter bli över matchade vi lärare ihop dessa med varandra. Denna matchning skedde genom att vi parade ihop de studenter som stod närmast varandra. Vi beaktade således inte studenternas kön, etniska tillhörighet, eventuella funktionsvariationer eller andra saker som är diskrimineringsgrundande.

2.2.1 Studentperspektiv

Studenter vill som regel inte särbehandlas. Detta gäller både positiv och negativ särbehandling. Det här innebär bland annat att du inte vill få mer eller mindre hjälp än dina medstudenter. Detta kan vara problematiskt, särskilt om studenten behöver mer hjälp än sina kamrater. Samtidigt har somliga studenter rätt till särskilt stöd, så som tolk eller inläst material.

Ett sätt att läsa detta på är genom att genomföra sin undervisning på ett stöttande sätt. På gymnasiet är det vanligt att elever med neuropsykologiska funktionsvariationer får extra tydliga instruktioner eller checklistor som de kan bocka av. Detta är något man som lärare kan ge alla studenter, eftersom detta är något som de allra flesta blir hjälpta av.

För mycket av den sortens hjälp skulle däremot kunna hindra studentens progression. För att inte det ska ske kan man bygga in det i sin undervisningsplan. Då kan man i början av undervisningen delge färdiga checklistor, för att senare tillsammans med studenterna skapa checklistor, så att de till sist kan skapa checklistorna själva.

2.2.2 Lärarperspektiv

Som lärare finns det en svårighet i att använda sina fördomar och förutfattande meningar om en grupp på ett konstruktivt sätt. Det är också svårt att göra skillnad på en fördom och en generaliserad verklighet. Ett exempel på detta är att om jag ser en student som använder en mac så tror jag inte att hen förstår

sig på filträd i datorn. Skulle studenten istället använda Linux så skulle jag förutsätta att studenten hade bra koll på filträd. Båda fallen är forutfattade meningar som jag har om studenter, där i alla fall den första har motbevisats.

Men den här typen av fördomar mot studenterna kan göra att man lägger undervisningen på fel nivå. Risken är stor att man lägger sig på en för lätt nivå när man pratar med en mac-användare eller en för hög nivå när man pratar med en Linux-användare. För att minimera problemet behöver man som lärare vara lyhörd och snabbt kunna växla mellan nivåer, och då helst på ett sätt som inte riskerar att skämma ut studenten eller dig själv.

2.2.3 Samhällsperspektiv

Att alla har lika stor rätt till utbildning slås fast av de mänskliga rättigheterna (Förenta nationerna, 1948, sida 7). Under kursens gång har detta problematiserats inom utbidlningar som leder till legitimation, exempelvis sjuksköterske- och receptiarieutbildningarna. Detta är utblidningar som är tänkta att leda till jobb där den anställde förväntas kunna utföra ett arbete, och där funktionsnedsättningar kan göra detta arbete omöjligt att utföra i praktiken. En blind sjuksköterska skulle ha svårt att göra bedömningar på exempelvis hudsjukdomar.

Om vi ser på högskolan som en kunskapsfabrik så är detta problematisk (Brandén, 2022, sida 274). Här har vi en student med rätten att studera men som inte kommer att kunna bli godkänd på flera av sina kurser, och därför inte kan nå examen. Ser vi istället högskolan som ett *samhällsförpliktigande universitet* så är den blinda studentens omöjlighet att bli legitimerad sjuksköterska inte ett problem. Studenten kan fortfrande ta till sig kunskap och växa som individ och samhällsmedborgare (Brandén, 2022, sida 283). Eventuellt skulle även en blind sjuksköterskestudent kunna bidra med insikter som andra studenter och lärare inte hade tänkt på annars.

2.3 Policy och styrdokument

De policy och styrdokument som används i det här papret är framförallt kursplanen samt utbildningsplanen.

2.3.1 Kopplingar till kursplan

I kursplanen står det att laborationerna ska bedöma målen:

1. förklara grundläggande begrepp inom objektorienterad programmering
2. redogöra för skillnader mellan imperativ (procedurell) och objektorienterad programmering, samt för- och nackdelar med olika programspråksparadigm,
3. förklara och ge exempel på olika sätt att organisera samlingar av objekt
4. konstruera mindre objektorienterade program där polymorfism nyttjas för att ge objekt generaliserat beteende (Malmö universitet, 2025a).

I det här momentet så testas framförallt mål 1 och det förbereder för att mål 4 ska kunna visas senare. Anledningen till att det inte riktigt kan visa på

färdigheter inom mål 4 är för att generalisering och polymorfism inte hunnits tas upp i kursen än.

2.3.2 Kopplingar till utbildningsplan

Ett av målen i utbildmningplanen är ”visa förmåga att planera och med adekvata metoder genomföra kvalificerade uppgifter inom givna ramar” (Malmö universitet, 2025b). Vilket studenterna jobbar mot i det här momentet.

2.3.3 Studentperspektiv

Momentet kan kopplas till både kursplanen och utbildningsplanen, detta har dock inte tydligt kommunicerats till studenterna. Istället har de fått information om att detta är ett obligatoriskt moment. Detta är oturligt, då utmärkt undervisning har tydliga mål för såväl lärare som studenter (Brandén, 2022, sida 142) Detta tvingar studenterna att blint lita på sina lärare och tro på att de har planerat momentet utifrån lärandemålen.

2.3.4 Lärarperspektiv

I skapandet av uppgiften har det som tidigare skrivits inte varit lärandemålen som direkt har format uppgiften. Istället har uppgiften formats efter tidigare moment. Nu har det råkat sig att uppgiften ändå knyter an till lärandemålen fyra, sju och åtta vilka ska examineras under det här momentet.

För lärarna som ska bedöma och handleda under momentet är det bra att veta vad uppgiftsförfattaren vill lära ut eller testa studenterna på (Brandén, 2022, sida 142). Det gör det lättare att handleda och styra studenterna i rätt riktning.

2.3.5 Samhällsperspektiv

Ur ett samhällsperspektiv är det viktigt att man har en uppfattning om vad studenterna kan när de är klara med en kurs eller utbildning. Detta är bland annat för att göra det lättare för studenter att flytta mellan universitet och för att lättare kunna anställas av näringslivet (Universitets- och högskolerådet, 2024). För att detta ska vara fallet behöver examinationsmomenten bedöma det som står i kursplanen.

3 Det nya kursmomentet

Det nya kursmomentet går ut på att studenterna får tre olika koder som ger samma resultat när de körs. Det tre programmen är skrivna i programspråket Java och hanterar stjärnssystem. När programmen körs kan användaren visa information om två förinlagda stjärnssystem, Sol samt Proxima Centauri, med deras omkretsande planeter. Användaren kan också lägga till nya stjärnsystem och fylla dessa med nya planeter som programmet sedan kan presentera.

Den första lösningen är programmerad med en imperativ lösning, alltså helt utan att använda objektorienterade lösningar. All information sparas i arrayer med strängar och kopplingen mellan stjärnor och planeter sker genom att en stjärna och dess planeter ligger på samma index i olika arrayer.

Den andra lösningen är en generell lösning med objektorientering. Det finns en klass för själva programmet och användargränssnittet, samt en klass som representerar en stjärna, och en klass som representerar en planet. Användargränssnittet har en array med stjärnor, och stjärnorna har varsin array med planeter. För att presentera information om stjärnorna och planeterna hämtar gränssnittet informationen direkt från objekten.

Den tredje lösningen följer *Model-View-Controller* designprincipen där användergränssnittet inte har någon direkt koppling till himlakropparna. Istället förmedlas informationen via en mellan-klass som hanterar programmets logik. Här har mellanklassen en array med stjärnor — som i sin tur har arrayer med planeter — och som instruerar användargränssnittet att presentera viss data.

Studenterna ska läsa igenom de tre programkoderna och köra programmen. De ska sedan diskutera skillnader och likheter i de tre lösningarna och presentera vilken lösning de tycker är bäst. Förhoppningen är att studenterna kommer fram till att det inte finns någon lösning som är objektivt bättre än de andra utan att alla tre har sina olika för- och nackdelar.

För att bli godkända så ska studenterna delta aktivt under hela passet, vilket kontrolleras löpande av närvarande lärare, samt kunna redogöra för olika för- och nackdelar mellan de olika lösningarna.

3.1 Förändringar

I detta avsnittet tas förändringarna av uppgiften upp samt hur dessa gör att uppgiften bättre kopplas till lärandemålen för både kursen och programmet.

3.1.1 Ändrad arbetsform

Uppgiften har gått från att studenterna jobbar två och två med att skriva kod, till att de sitter i grupper om tre och diskuterar kod. Det är tre olika anledningar till att denna förändring har gjorts.

Den första anledningen är att de skriver kod under de övriga laborationspassen. Under dessa passen kan de be om hjälp utan att det finns en press om att uppgiften ska bedömas. Även om det var tillåtet att be om hjälp i den tidigare uppgiften var det få som vågade göra det eftersom det var en examinerande uppgift. Vi kontrollerar även deras förmåga att skriva kod med andra examinationsuppgifter, så att kolla det nu är överflödigt. Det var dessutom hyfsat enkelt att be en ”smart kompis” på internet, generativ AI, att generera en lösning. Ber man Copilot365 att jämföra koderna och ge för- och nackdelar med de tre olika programmen får man tyvärr ett bra svar även med den nya uppgiften.

Den andra anledningen är rättningsbördan. Vi upptäckte att redovisningarna tog väldigt lång tid när vi genomförde det ursprungliga momentet. Detta trots att programmet var självrättande i den mån att studenterna fick ett meddelande om koden gjorde det vi förväntade oss. Att gå igenom deras lösningar och se så att de även förstod vad de hade gjort tog så pass lång tid att passet drog över med ungefär en 45 minuter. Att istället kunna gå runt och lyssna på studenterna under passet för att sedan höra deras slutsatser i slutet kan gå betydligt snabbare.

Den tredje och största anledningen till att byta arbetsform är att jag anser att studenterna behöver få se bra kod. De behöver få se korrekta lösningar med en kodstandard vi förväntar oss att de följer. Detta ger studenterna en tydlig

förebild vilket kan gynna deras fortsatta inlärning (Elmgren & Olding, 2025, s. 91) . Får studenterna inte se vad vi menar är en bra lösning är det svårt för dem att själva avgöra kvalitén i sina egna och kamraters lösningar vilken kan minska värdet av framtida kamratgranskning (Elmgren & Olding, 2025, sida 280). Vi vill även att de börjar fundera på sina val av lösningar. Genom att diskutera tre olika alternativ är förhoppningen att studenterna får en medvetenhet om att det finns olika sätt att lösa ett problem och att de framgent gör aktiva val av metoder för att lösa sina egna problem.

3.1.2 Förtydliganden i uppgiften

En förändring i hur uppgiften presenteras är att det nu finns med ett stycke om den pedagogisk idéen bakom momentet. Att låta studenterna veta att det finns en medveten plan kring momentets upplägg stärker uppgiften på två sätt.

Först försäkras studenten om att det finns en plan och att det inte är ett godtyckligt moment som finns med ”bara för att”. Det kan ge en tillit till undervisningen och lugna studenten med att om hen följer lärarens plan och instruktioner så kommer hen att komma vidare i sitt lärande (Brandén, 2022, sida 123).

Den andra sättet detta stärker uppgiften på är att det visar studenten hur momentet kopplas till lärandemålen och varför vi som lärare anser att detta är en bra metod för att nå dit. Detta är särskilt viktigt när man avviker från vad många ser som det traditionella sättet att lära sig kodning, nämligen *learning by doing*. Det är synnerligen viktigt då jag själv ofta repeterar för studenterna att knacka kod är nödvändigt för att bli bättre på att koda. Att nu förklara varför uppgiften avviker från ett etablerat mönster kan minska motståndet från studentera att ta sig an uppgiften ordentligt (Brandén, 2022, sida 160).

3.2 Konstruktiv länkning

Att jämföra olika program är något som kan testas på en tentamen. Där kan studenterna få två eller fler olika koder som ska jämföras där de sedan ska diskutera för och nackdelar med de olika implementationerna. I inläming två ställs också studenterna inför designbeslut och förhoppningen är att detta moment har förberett dem för att fatta det beslutet.

Faller utfallet ut väl i det här momentet skulle följande workshopar i kursen kunna följa samma stuk, där de får färdig kod som ska analyseras och behandlas på någotvis. Det skulle även vara möjligt att inlämningsuppgifterna ska redovisas på liknande sätt där studenterna får läsa igenom varandras lösningar och diskutera sina olika designval. Annars, som det ser ut nu, är där inte någon nämnvärd konstruktiv länkning till övriga moment i kursen.

Om man inte gör andra ändringar i övriga kursmoment hamnar vi i en sits där uppgiften i sig är en bättre uppgift, men den konstruktiva länkningen minskar.

3.3 Likabehandling

I den tidigare arbetsformen har studenter med dålig datorvana, dyslexi, eller andra hinder som gör att de har svårt att skriva på ett tangentbord missgynnats.

När text ska skapas på en begränsad tid kan det orsaka en stress och få studenter att underprestera jämfört med om uppgiften varit utformad på ett annat sätt.

Den nya varianten av uppgiften missgynnar fortfarande dyslektiker eftersom det blir mer att läsa. Att få kod uppläst är problematisk av framförallt två anledningar:

1. Det är *konstiga* ord och ingen naturlig meningsbyggnad. Ofta förkortas ord något godtyckligt och språk kan blandas. Exempelvis är uttrycket ”`System.out.println(arr[i]/5 == 3);`” ett fullt logiskt och syntaktiskt korrekt uttryck i Java. Men för en textuppläsare är det svårt att läsa upp korrekt. Detta kan exempelvis testas i Microsoft Word som har en textuppläsningsfunktion. Är Word inställt på svenska läser den upp ”System out println arr i snedstreck fem lika med lika med tre”. Medan en programmerare skulle kunna läsa det som ”Skriv ut: är arr på plats i, delat på fem lika med tre?”.

2. Textuppläsning tar tid. De tar som regel 30 % längre tid att läsa en text högt än vad det tar för en genomsnittlig person att läsa en text tyst (Brysbaert, 2019). Men med den rapporterade minskade läsförståelsen kanske detta är ett glapp som numera minskat (Viktorsson & Nyrén, 2024).

Den tidigare uppgiften var — inklusive kod som behövde läsas — tio sidor lång. Den nya uppgiften är — med koden inräknad — ca 30 sidor lång. Detta är en markant ökning av text. Däremot har producerandet av text strukits. Detta kan kompensera för den större textmassan som ska läsas. Även om det totala antalet sidor som ska läsas har ökats har uppgiftsbeskrivningen dragits ner från sex sidor till två och en halv sidor.

Att läsa kod kan inte riktigt likställas med att läsa en löpande text. En programkod som huvudsakligen är skriven på engelska kan snarare likställas med strukturerad engelska (*eng. Structured English*). En sida programkod innehåller ca 200 ord (detta är taget från koden studenterna får i den här uppgiften). Varje rad på en sida är alltså förhållandevis kort vilket underlättar för personer med lässvårigheter (Dyslexia Scotland, 2025). Jämförelsevis har det här papret cirka 500 ord per sida.

Den nya utformningen kan försvåra för studenter som inte har svenska som modersmål (Bälter m. fl., 2023). Ett krav för att komma in på utbildningen 2025 är ett godkänt betyg i Svenska 3 eller Svenska som andraspråk 3 (Malmö universitet, 2025b). Att föra en ämnesteknisk diskussion är svårt om man har låga ämneskunskaper — vilket de flesta studenter har i början av sin utbildning — det är ännu svårare om man också har svaga språkkunskaper. Detta är något som studenterna behöver öva på vilket de får möjlighet till i den här uppgiften.

3.4 Policy och styrdokument

Liksom tidigare version av momentet så undersöks den nya versionen jämtemot kursplanen och utbildningsplanen.

3.4.1 Kopplingar till kursplan

De lärandemål i kursplanen som momentet är menat att, framförallt, träna är

- Efter avslutad kurs skall studenten kunna: redogöra för skillnader mellan imperativ (procedurell) och objektorienterad programmering, samt för- och nackdelar med olika programspråksparadigm.
- Efter avslutad kurs skall studenten kunna: motivera val av lämpliga kontrollstrukturer och klasser vid konstruktion av ett objektorienterat program.

Ett problem här är att som kursplanen är skriven idag så ska laborationerna inte bedöma det första utav lärandemålen. Det här passet förbereder däremot studenterna för att kunna visa den kunskapen när de kommer till den skriftliga tentamen som ska bedöma det lärandemålet (Malmö universitet, 2025a).

Uppgiften skapar också en brygga mellan den tidigare kursen CD100 och den här kursen genom att visa en lösning som följer principerna från den kursen, och visar en lösning som följer principerna i den här kursen. Den kan även ses som förberedande för att i andra moment nå andra kursmål.

3.4.2 Kopplingar till utbildningsplan

Ett av målen för civilingenjörsutbildningen på Malmö universitet är att studenten ska utveckla sin ”förmåga att göra självständiga och kritiska bedömningar” (Malmö universitet, 2025b). Den nya utformningen av uppgiften riktar in sig på det här målet genom att här låta studenterna få kritiskt jämföra olika kodstycken.

Enligt högskoleförordningen ska ”studenten visa kunskap om det valda teknikområdets vetenskapliga grund och beprövade erfarenhet samt insikt i aktuellt forsknings- och utvecklingsarbete” (HF, 2025). I uppgiftens nya utformning får studenten ta del av tre olika paradigmer inom programmering som är en del av proffessionens gemensamma erfarenhet.

Vad samhället förväntar sig av en färdigutbildad civilingenjör framgår i högskoleförordningen. Där framgår bland annat att en civilingenjör ska kunna ta hänsyn till samhälleliga aspekter, samt miljö- och arbetsmiljöaspeketer i sitt arbete (HF, 2025). Att kunna värdera sina lösningar mot vad en situation kräver leder till kod som är lättare att underhålla och vidareutveckla. Det leder till mjukvara som fungerar en större del av tiden och som lättare kan lagas när det behövs.

Att känna till olika lösningar och paradigm gör också att det blir lättare att arbeta med olika projekt, olika projektgrupper, och mot olika mål. På ett sätt kan man kalla en utvecklare med en bred kunskap av olika paradigm för ”mångkulturerad”, i den mening att hen är kunnig och delaktig inom flera olika programmeringskulturer.

4 Hur AI har använts i den här texten

I skrivprocessen för den har rapporten har AI använts för att formatera .bib-filen med referenser korrekt. Exempelvis har Copilot365 använts för att få fram vilka fält som ska fyllas i när man refererar till lagstiftning. Copilot365 har även använts för att kontrollera om texten är sammanhängande, om det är några brister i det framförda resonemanget, samt om språkbruket är konsekvent.

Referenser

Biggs, J., Tang, C., & Kennedy, G. (2022). *Teaching for quality learning at university* (5. utg.). McGraw-Hill.

Brandén, H. (2022). *Lärande, undervisning och arbete inom högre utbildning* (1. utg.). Sanoma Utbildning.

Brysbaert, M. (2019). How many words do we read per minute? a review and meta-analysis of reading rate. *Journal of Memory and Language*, *109*, 104047. https://doi.org/10.1016/j.jml.2019.104047

Bälter, O., Kann, V., Mutimukwe, C., & Malmström, H. (2023). English-medium instruction and impact on academic performance: A randomized control study. *Applied Linguistics Review*, *15*, 2373–2396. https://doi.org/10.1515/applirev-2022-0093

Dyslexia Scotland. (2025). Dyslexia-friendly typed content. Hämtad 3 mars 2026, från https://dyslexiascotland.org.uk/wp-content/uploads/2025/09/formats.pdf

Elmgren, M., & Olding, A.-S. (2025). *Universitetspedagogik* (4:1). Studentlitteratur.

Förenta nationerna. (1948). Universal declaration of human rights. Hämtad 9 mars 2026, från https://www.un.org/sites/un2.un.org/files/2021/03/udhr.pdf

Högskoleförordningen (2025). Hämtad 6 februari 2026, från https://rkrattsbaser.gov.se/sfst?bet=1993:100

Malmö universitet. (2025a). Kursplan för CD102A: Objektorienterad programmering. Hämtad 5 februari 2026, från https://utbildningsinfo.mau.se/kurs/kursplan/sv/a3294619-8619-11ef-9561-318924f24352/20252

Malmö universitet. (2025b). Utbildningsplan för Civilingenjör Datateknik. Hämtad 4 februari 2026, från https://utbildningsinfo.mau.se/program/utbildningsplan/sv/7c5f2e73-bdfe-11ef-a8c1-e278cbfa4af0/20252

Statistiska centralbyrån. (2025). Examensfrekvens för nybörjare på program mot civilingenjörs- och högskoleingenjörsexamen uppföljningsläsåren 2019/20–2023/24, efter SUN-inriktning (3-siffernivå). Hämtad 11 mars 2026, från https://www.scb.se/hitta-statistik/statistik-efter-amne/utbildning-samt-forskning-inom-hogskolan/hogskolevasende/genomstromning-och-resultat-i-hogskoleutbildning-pa-grundniva-och-avancerad-niva/#_Tabellerochdiagram

Universitets- och högskolerådet. (2024). Bologna processen - det europeiska området för högre utbildning. Hämtad 9 mars 2026, från https://www.uhr.se/internationella-mojligheter/Bolognaprocessen/

Universitets- och högskolerådet. (2025). Antagning till högre utbildning höstterminen 2025. Hämtad 11 mars 2026, från https://www.uhr.se/globalassets/_uhr.se/publikationer/2025/antagning-till-hogre-utbildning-varterminen-2026.pdf

Universitets- och högskolerådet. (2026). Antagningsstatistik. Hämtad 11 mars 2026, från https://www.uhr.se/studier-och-antagning/antagningsstatistik/resultatsida/?astasearchperiod=HT25&astasearchfor=data

Viktorsson, J., & Nyrén, J. (2024). *Skånska toppstudenter kan inte läsa böcker: "Orkar inte"*. Sydsvenskan. Hämtad 5 februari 2026, från https://www.sydsvenskan.se/skane/skanska-toppstudenter-kan-inte-lasa-bocker-orkar-inte/

Ämnesplan Fysik (2010). Hämtad 26 februari 2026, från https://www.skolverket.se/undervisning/gymnasieskolan/program-och-amnen-i-gymnasieskolan/hitta-program-amnen-och-kurser-i-gymnasieskolan-gy11/hitta-program-och-amnesplaner-i-gymnasieskolan-gy11#/search/subjects/FYS?v=3#FYSFYS01a

A Antagna och sökande hösten 2025

Sökande och antagna efter andra urvalet hösten 2025

	Män	Kvinnor	Totalt
Antal sökande	73149	33487	106636
Andel sökande	69 %	31 %	100 %
Antal antagna	5663	2182	7845
Andel antagna	72 %	28 %	100 %

Källa: (Universitets- och högskolerådet, 2026)

Sökningen som gjordes var program som innehåller ”data” som nyckelord. Detta är en bred sökning som kan få med utbildningar som inte har programmering. Dessutom skulle utbildningar som saknar en ”data”-tagg men som ändå innehåller programmering saknas. Men detta bortfall tordes vara försumbart, likaså den extra datan.

B Genomströmning på program med inriktning Elektronik, datateknik och automation

	Män	Kvinnor
Antagna	10171	2040
Examen	3556	694
Andel	35 %	34 %

Tabellen visar hur många män respektive kvinnor tagit examen från ett Civil- eller högskoleingenjörsprogram med en inriktning mot elektronik, datateknik, eller automation som inom tre år efter att deras nominella studietid har gått. Datan visar studenter som påbörjat sina ingenjörsstudier mellan åren 2012 och 2018 (Statistiska centralbyrån, 2025).

C Tidigare version av uppgiften

CD102A Laboration L5

Syfte

Den här laborationen genomförs i workshopformat. Laborationens syfte är att applicera det som har gåtts igenom om klassdiagram och objekt. Du kommer att behöva veta hur du skapar ett klassdiagram och hur du från ett klassdiagram sedan skapar flera klasser.

Lösningarna diskuteras med andra studenter för att öva på att presentera lösningar och skaffa en vana av detta inför kommande laborationer och inlämningsuppgifter som ska redovisas. För- och nackdelar med lösningar diskuteras som en del av detta för att öva på att granska sin egen lösning och för att förstå andras lösningar.

Följande tas upp i denna labb

- Klassdiagram
- Klass, objekt, attribut och operationer
- Associationstyper

Redovisning

Laboration L5 kräver godkänt deltagande i schemalagt tillfälle. **Godkänd laboration L5 krävs för godkänt betyg G på provkod 2501 — Laborationer.** Du ska alltid vara beredd på att kunna visa legitimation vid redovisning/genomförande.

De studenter som deltar aktivt i laborationen på campus och deltar under hela passet registreras där via närvarolista. **För godkänt aktivt deltagande krävs att**

- **ingen grupp som arbetar tillsammans är mer än tre personer**
    - **Enbart en grupp kommer att tillåtas bestå av tre personer utifall att ett udda antal studenter deltar vid laborationen**
- **alla i gruppen ska kunna redovisa alla delar av uppgiften**

Det är viktigt att du skriver på närvarolistan när du kommer och igen vid godkänd redovisning. Underskrifter från närvarolistor registreras sedan av lärare på Canvas.

Om du inte får en notering om godkänd i Canvas senast två arbetsdagar efter att du genomförde momentet och anser att du då fick besked om att du var godkänd bör du kontakta ansvarig

lärare för att kontrollera att något inte blivit fel. Om du kontaktar lärare en längre tid efter att du genomförde momentet kan din närvaro kanske inte längre bekräftas och du kan behöva göra om momentet.

Det garanteras endast två omtillfällen att genomföra L5 på om man inte kan delta i det ordinarie tillfället. Omtillfällena finns utlagda på kursens schema i Kronox.

Förberedelser

Den här laborationen är direkt kopplad till det innehåll som specifikt finns i föreläsning F4 och F5. För att kunna genomföra laboration L5 behöver studenten ha bekantat dig med begrepp från alla tidigare föreläsningar. Man bör även ha gjort tidigare laborationer.

Om lösningar

Det finns ibland vissa direkta fel eller olämpliga lösningar men det finns alltid flera olika ”rätta” lösningar. Olika lösningar har olika fördelar och nackdelar. Om någon annans lösning ser annorlunda ut än din lösning behöver det inte betyda att någon av er har fel. Det kan vara så att ni bara har två olika lösningar med olika fördelar och nackdelar och inget är direkt fel – det är bara olika.

Uppgiften

Nedan följer tre klassbeskrivningar. Ni ska utifrån dessa skapa de tre klasserna **Galaxy**, **Starsystem** och **Planet**. Ni ska även skapa ett motsvarande klassdiagram för de tre klasserna samt rita ut relationerna mellan de tre klasserna. Ni rekommenderas att först skapa klassen **Planet**, sedan klassen **Starsystem** och sist klassen **Galaxy**. I den här uppgiften är *inga* bibliotek tillåtna att använda.

Redovisning

När ni är redo att redovisa informerar ni lärare och/eller assistenter enligt den metod som angivits vid genomgången. För att redovisa ska den **Main**-klass ni fått ha skrivit ut att ni är redo att presentera (ert program kommer då att skriva ut: *Present your solution*). Under redovisningen ska ni presentera både ert klassdiagram samt er kod till de tre klasserna. Alla medlemmar i gruppen ska kunna svara på alla frågor. Det är gruppens ansvar att alla i gruppen kan gruppens lösning. Redovisningen sker i klassrummet, inför den lärare/assistent som är tillgänglig.

Klassen Planet

Klassen **Planet** modellerar en planet på en grundläggande nivå.

Attribut

Den innehåller två privata attribut, strängen *name* samt double:n *mass*. (Massorna i programmet kommer att anges i enheten $M_{\oplus}$ *jordmassa*).

Konstruktorer

Klassen ska ha två (2) konstruktorer. En (1) *default*-konstruktor som som anropar den andra konstruktorn *Planet(name:String, mass:double)* med default värdena ”*Unknown*” och *0*. Den andra konstruktorn ska ta emot en sträng som ska sparas i *name* och en double som ska sparas i *mass*.

Operationer

Klassen **Planet** ska ha fem (5) stycken övriga publika operationer.

getName() : String

Den här operationen ska returnera attributet *name*.

getMass() : double

Den här operationen ska returnera attributet *mass*.

setName(newName : String)

Den här operationen ska tilldela värdet i *newName* till attributet *name*.

setMass(newMass : double)

Den här operationen ska tilldela värdet *newMass* till attributet *mass*.

toString() : String

Den här operationen ska returnera attribut *name*.

Klassen Starsystem

Klassen **Starsystem** håller reda på en drös med planeter som kretsar runt en stjärna.

Attribut

Klassen **Starsystem** har två (2) private attribut. Dessa är strängen *name* samt **Planet**-arrayen *planets*.

Konstruktorer

Den här klassen har tre (3) olika konstruktorer. Den första ska vara en *default*-konstruktor som anropar någon av de andra två konstruktorerna och sätter attributet *name* till ”*Unknown*” samt arrayen *planets* till en tom **Planet**-array med ett valfritt antal tomma platser. Den andra konstruktorn ska ta emot en sträng *name* som ska sparas i attributet *name*. Även till denna konstruktorn ska en tom **Planet**-array skapas. I den sista kosntruktorn så ska både en sträng för namnet tas emot samt en array av typen **Planet**. Dessa planeter ska sparas i attributet *planets*.

Operationer

Klassen **Starsystem** har fyra (5) övriga operationer som alla är *public*.

getName() : String

Den här operationen ska returnera attributet *name*.

calculateMass() : double

Den här operationen ska returnera summan av alla attributen *mass* som finns i arrayen *planets*. Här kommer du att behöva loopa över arrayen *planets*. Tänk på att det kan förekomma tomma platser i slutet av arrayen.

getPlanet(planetName : String) : Planet

Den här operationen ska returnera det **Planet**-objekt i arrayen *planets* som har värdet *planetName* på sitt attribut *name*. Om ingen planet med det namnet hittas ska operationen returnera *null*. Här kommer du att behöva loopa över arrayen *planets*.

addPlanet(planetName : String, planetMass : double)

Den här operationen ska ta emot en sträng *planetName* samt en double *planetMass* och skapa ett nytt **Planet**-objekt med namnet *planetName* och massan *planetMass*. Det nya objektet ska sedan läggs till på den första lediga platsen i arrayen *planets*. Om

arrayen är full så ska programmet utöka arrayen för att sedan lägga till planeten. (Detta görs i Java genom att man skapar en ny array med fler platser, sparar över alla värden från den gamla arrayen i den nya (genom att loopa över original-arrayen) och sedan spara över hela den gamla arrayen med den nya arrayen genom en enkel tilldelning.)

toString() : String

Den här operationen ska returnera en sträng uppbyggd på följande vis:
Stjärnans namn, planet 1:s namn, planet 2:s namn, …
Exempelvis:
Sol, Mercury, Venus, Earth

Klassen Galaxy

Klassen **Galaxy** håller reda på en drös med stjärnsystem med planeter.

Attribut

Klassen **Galaxy** har två (2) private attribut. Dessa är strängen *name* och samt **Starsystem**-arrayen *starsystems*.

Konstruktorer

Den här klassen har en (1) konstruktor som tar emot strängen *name* som ska sparas i attributet *name*. I konstruktorn ska även arrayen *starsystems* initialiseras med max fem (5) tomma platser.

Operationer

Utöver konstruktorn har klassen fem (5) operationer som alla är *public*.

getName() : String

Den här operationen ska returnera attributet *name*.

getPlanet(planetName : String) : Planet

Den här operationen ska göra en linjär sökning genom alla element i arrayen *starsystems* och returnera den planet med ett namn som matchar strängen *planetName*. Om ingen planet med rätt namn hittas ska operationen returnera *null*.

getStarsystem(systemName : String) : Starsystem

Den här operationen ska göra en linjär sökning genom alla element i arrayen *starsystems* och returnera det stjärnsystem med ett namn som matchar strängen *systemName*. Om inget system med rätt namn hittas ska operationen returnera *null*.

createStarsystem(starName : String, planets : Planet[])

Den här operationen ska skapa ett nytt objekt av typen **Starsystem** och lägga till det i arrayen *starsystems*. Om arrayen är full så ska programmet utöka arrayen för att sedan lägga till systemet. (Detta görs i Java genom att man skapar en ny array med fler platser, sparar över alla värden från den gamla arrayen i den nya (genom att loopa över original-arrayen) och sedan spara över hela den gamla arrayen med den nya arrayen genom en enkel tilldelning.)

toString() : String

Den här operationen ska returnera attributet *name*.

D Uppdaterad version av uppgiften

Workshop 1

CD102A

HT26

1 Inledning

I den här workshopen kommer ni att få se tre olika lösningar på samma problem. Alla tre lösningarna har för- och nackdelar. Under den här workshopen kommer ni att diskutera dessa för att komma fram till skillnader i designval (*imperativ*, *objektorientering*, samt *Model-View-Controller*).

2 Programmets funktion

Ni har fått tre versioner av samma program, som alla gör samma sak. Programmet håller information om stjärnor och planeter som kretsar kring stjärnorna. De två stjärnssystemen *Sol* och *Proxima Centauri* är förinlagda i programmet med sina respektive omkretsande planeter.

I programmet är det möjligt att se information om stjärnorna och planeterna. Informationen du kan se om stjärnorna är deras namn samt antalet planeter som kretsar runt dem, och informationen om planeterna är deras namn och massa. Massan mäts i *jordmassor*.

Under körning är det möjligt att lägga till nya stjärnor och nya planeter som kretsar kring stjärnorna. En planet kan bara kretsa kring en stjärna och varje planet är distinkt i.e. inga planeter är delade mellan stjärnorna. Det är möjligt att stjärnor har flera identiska planeter som kretsar kring dem.

3 Bakgrund

I den här kursen finns tre provkoder som behöver avklaras för att bli godkänd på kursen.

2501 Laborationer

2502 Inlämningsuppgifter

2503 Tentamen

Under provkod 2501 examineras bland annat förmågan att konstruera objektorienterade program.

3.1 Lärandemål

Uppgiften kopplas till följande lärandemål i kursplanen:

- använda diagram som verktyg för att utforma, kommunicera och dokumentera objektorienterad design,
- motivera val av lämpliga kontrollstrukturer och klasser vid konstruktion av ett objektorienterat program.

3.2 Pedagogisk idé

Ett vanligt angreppssätt på en programmeringsuppgift är att genast ge sig in i att koda och lösa problem allt eftersom de uppstår. Detta fungerar som regel i mindre projekt där komplexiteten är låg. Ju större projekt, och ju fler som jobbar på projektet, desto mer krävs förberedelser. (Jämför förberedelserna för att bre en macka med att laga en tre-rätters middag.) För att kunna genomföra ett större projekt behöver man ha en tydlig plan, och för att ha en tydlig plan behöver man ha kunskap om olika lösningsmetoder. I den här uppgiften stöter ni på tre olika lösningsmetoder för att lösa samma problem.

Vi vill även att ni får se ”bra” kod, med vad vi menar är acceptabel dokumentation. Det är annars vanligt att nybörjare gör något som fungerar och löser problemet men att det görs omständigt och rörigt. Vi vill alltså med de här tre programmen ge er något av en mall för hur variabler kan namnges, hur ni kan skriva JavaDocs, och allmänt hur kod kan struktureras.

4 Uppgiften

- Ni ska under workshopen diskutera de tre lösningarna i grupp och bestämma deras för- respektive nackdelar.
  - Inledningsvis ska ni läsa in er på hur de olika lösningarna fungerar.
  - Ni förväntas att kunna förklara varje metod i varje lösning.

- Ni ska skapa klassdiagram över lösningarna B och C. Klassdiagramen ska visa klassernas
  - namn,
  - attribut,
  - relationer till varandra.

5 Redovisning

Uppgiften redovisas tidigast efter 60 minuter. Under redovisningen kommer gruppen att förhöras på sin förståelse av de tre lösningarna. Ni ska visa på att ni ser för- och nackdelar med alla tre lösningar, samt att ni har gjort korrekta klassdiagram. Dessutom krävs det att alla i gruppen deltar aktivt i diskussionerna under workshopen.